

A)

randomSearch(Array A, int n, int x):

- Create an empty set called check
- Create a random object
- While checked doesn't contain all n indices:
 - Pick a random index i between 0 and n-1
 - If A[i] equals x:
 - Return i
 - Add i to check
- Return -1 if not found

B)

Case	Expectation
1 match	n
K matches	n / K
0 matches	$\Theta(n \log n)$

E,F,G)

determinSearch(array A, int n, int x):

- For i starting at 0, going to n-1:
 - If A[i] is x:
 - Return i
- Return -1 if not found

Case	Average	Worst Case
1 Match	$\Theta(n)$	$\Theta(n)$
K matches	$\Theta(n / (k + 1))$	$\Theta(n - k)$
0 mathces	$\Theta(n)$	$\Theta(n)$

H)

scrambleSearch(array A, int n , int x):

- For i starting at n-1 going to 1:
 - Pick a random index j between 0 and i
 - Swap A[i] and A[j]
- Return determinSearch(A, n, x)

Case	Expected	Worst Case
$K = 0$	$\Theta(n)$	$\Theta(n)$
$K = 1$	$\Theta(n)$	$\Theta(n)$
$K > 1$	$\Theta(n / (k + 1))$	$\Theta(n - k)$

l)

I would use deterministic search because random search can randomly pick the same index multiple times. Scramble search just shuffles first and then does the exact same linear scan each time. Deterministic search is just a loop that checks and is always $\Theta(n)$ so its the most reliably consistent